

GUÍA DE DESARROLLO

NestJS Backend

TypeScript · TypeORM · PostgreSQL · JWT · Swagger

Guía paso a paso basada en el proyecto OCSO.
Cubre cada tecnología en el orden exacto en que se aplica
al construir una API REST profesional desde cero.

Stack utilizado en este proyecto

TypeScript

Lenguaje base

NestJS

Framework

TypeORM

ORM

PostgreSQL

Base de datos

JWT

Autenticación

bcrypt

Seguridad

Swagger

Documentación

Multer

Archivos

0 Índice de Contenidos

- 01 TypeScript — Fundamentos base
- 02 NestJS — Arquitectura y estructura
- 03 TypeORM — Base de datos con PostgreSQL
- 04 @nestjs/config — Variables de entorno
- 05 class-validator — Validación de DTOs
- 06 bcrypt — Hasheo de contraseñas
- 07 JWT — Autenticación con tokens
- 08 Guards — Protección de rutas
- 09 Decoradores personalizados
- 10 Pipes — Transformación de parámetros
- 11 Swagger — Documentación automática
- 12 Multer — Carga de archivos

Orden paso a paso para nuevos proyectos

01 TypeScript

El lenguaje base del proyecto

¿Qué es?

TypeScript es un superset de JavaScript que agrega tipado estático. Todo el código de NestJS se escribe en TypeScript. Antes de ejecutarse, se compila a JavaScript plano.

Conceptos clave usados en este proyecto

Concepto	Para qué sirve	Dónde aparece en el proyecto
enum	Define un conjunto de valores con nombre	ROLES (Admin, Manager, Employee)
class	Estructura base de entidades y DTOs	Employee, User, CreateEmployeeDto
interface	Define la forma de un objeto	Tipos de retorno y parámetros
declare	Re-declara propiedad heredada	DTOs que extienden la entidad
async/await	Maneja operaciones asíncronas	Todos los métodos de servicio
generics T	Tipos reutilizables (Repository<T>)	Repository<Employee>, Repository<User>

Ejemplo: enum de roles

```
■ src/auth/constants/roles.constants.ts
export enum ROLES {
  ADMIN = 'Admin',
  MANAGER = 'Manager',
  EMPLOYEE = 'Employee'
}
```

Ejemplo: declare en un DTO

Los DTOs extienden la entidad para heredar sus propiedades, pero usan **declare** para re-tiparlo con validaciones sin crear una nueva propiedad en memoria:

■ `src/employees/dto/create-employee.dto.ts`

```
export class CreateEmployeeDto extends Employee {  
  @IsString()  
  @MaxLength(30)  
  declare employeeName: string; // 'declare' = solo tipo, sin nueva propiedad  
  
  @IsEmail()  
  declare employeeEmail: string;  
}
```

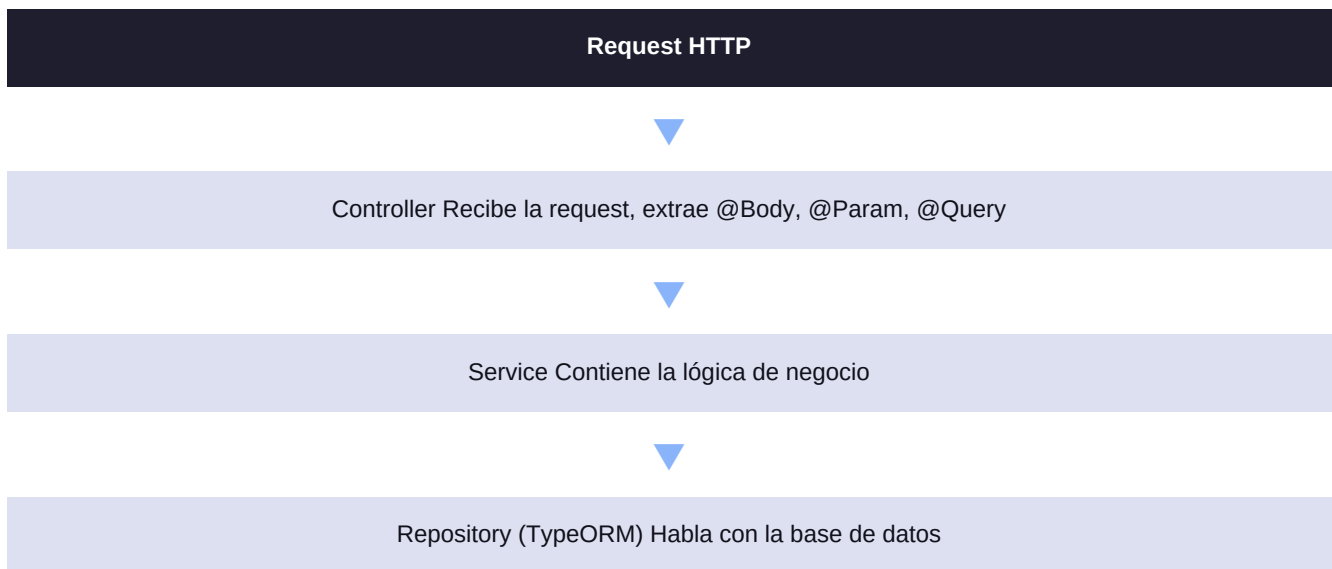
02 NestJS

Framework principal — Módulos, Controladores y Servicios

¿Qué es?

NestJS es un framework de Node.js construido sobre Express. Organiza el código en **módulos**, siguiendo el patrón MVC adaptado a APIs. Usa decoradores de TypeScript para declarar rutas, inyectar dependencias y configurar comportamientos.

El patrón fundamental: Module → Controller → Service



Decoradores del Controller

	Nivel	Para qué	
ta')	Clase	Define el prefijo de todas las rutas del controlador	
	@Post()	Método	Define el verbo HTTP y la sub-ruta
	Parámetro	Extrae el cuerpo JSON del request	
ore')	Parámetro	Extrae un segmento de la URL /ruta/:id	
re')	Parámetro	Extrae un query param /ruta?pagina=1	
e())	Parámetro	Extrae el archivo subido (con Multer)	

Ejemplo completo de un módulo

■ src/employees/employees.module.ts

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { EmployeesService } from './employees.service';
import { EmployeesController } from './employees.controller';
import { Employee } from './entities/employee.entity';

@Module({
  imports: [TypeOrmModule.forFeature([Employee])], // registra la entidad
  controllers: [EmployeesController],
  providers: [EmployeesService],
})
export class EmployeesModule {}
```

Ejemplo de Controller con rutas

```
■ src/employees/employees.controller.ts (simplificado)
@Controller('employees')
export class EmployeesController {
  constructor(private readonly employeesService: EmployeesService) {}

  @Post() // POST /employees
  create(@Body() dto: CreateEmployeeDto) {
    return this.employeesService.create(dto);
  }

  @Get() // GET /employees
  findAll() {
    return this.employeesService.findAll();
  }

  @Get('/:id') // GET /employees/:id
  findOne(@Param('id') id: string) {
    return this.employeesService.findOne(id);
  }

  @Patch('/:id') // PATCH /employees/:id
  update(@Param('id') id: string, @Body() dto: UpdateEmployeeDto) {
    return this.employeesService.update(id, dto);
  }

  @Delete('/:id') // DELETE /employees/:id
  remove(@Param('id') id: string) {
    return this.employeesService.remove(id);
  }
}
```

03 TypeORM

Mapeo de objetos a tablas de PostgreSQL

¿Qué es?

TypeORM es un ORM (Object-Relational Mapper) que permite definir las tablas de la base de datos como clases TypeScript, y hacer operaciones CRUD sin escribir SQL directamente.

3.1 Configuración global en AppModule

```
■ src/app.module.ts
TypeOrmModule.forRoot({
  type: 'postgres',
  host: process.env.host,
  port: +(process.env.port || '5432'),
  username: 'postgres',
  password: process.env.pass,
  database: process.env.name,
  autoLoadEntities: true, // carga entidades registradas con forFeature()
  synchronize: true, // crea/modifica tablas automáticamente (solo dev)
})
```

■ synchronize: true es solo para desarrollo. En producción debe ser false y se usan migraciones para no perder datos.

3.2 Decoradores de columnas más usados

Decorador	Para qué
@Entity()	Marca la clase como tabla en la BD
@PrimaryGeneratedColumn('uuid')	ID único auto-generado tipo UUID
@PrimaryGeneratedColumn('increment')	ID numérico auto-incremental (1, 2, 3...)
@Column('text')	Columna de texto
@Column('float')	Columna numérica decimal
@Column('simple-array')	Guarda un array como texto separado por comas
@Column({ unique: true })	Valor único, no puede repetirse

@Column({ nullable: true })	La columna puede estar vacía (null)
@Column({ default: 'valor' })	Valor por defecto si no se envía

3.3 Relaciones entre entidades

Las relaciones definen cómo se conectan las tablas entre sí. Siempre se declaran en ambas entidades y el lado que tiene **@JoinColumn** es el que guarda la clave foránea (FK).

Decorador	Relación	Ejemplo en el proyecto
@OneToOne	1 registro ↔ 1 registro	User ↔ Manager, User ↔ Employee
@ManyToOne	Muchos registros → 1 registro	Employee → Location (muchos employees en una location)
@OneToMany	1 registro → Muchos registros	Location → Employee[] (una location tiene muchos employees)
@JoinColumn	Define cuál lado tiene la FK	Va en el lado 'dueño' de la relación

Ejemplo de relaciones en la entidad Employee

```

■ src/employees/entities/employee.entity.ts
@Entity()
export class Employee {
  @PrimaryGeneratedColumn('uuid')
  employeeId: string;

  @Column('text')
  employeeName: string;

  @Column({ type: 'text', nullable: true })
  employeePhoto: string;

  // Muchos employees pertenecen a una Location
  @ManyToOne(() => Location, (location) => location.employees)
  @JoinColumn({ name: 'locationId' }) // FK en esta tabla
  location: Location;

  // Un employee tiene un User (login)
  @OneToOne(() => User)
  @JoinColumn({ name: 'userId' })
  user: User;
}

```

3.4 Métodos del Repository

Método	Para qué	Retorna
.find()	Trae todos los registros	Entity[]
.findOne({ where: {...} })	Busca uno por condición	Entity null
.findOneBy({ campo: valor })	Busca por campo directo	Entity null
.findBy({ campo: valor })	Trae varios por campo	Entity[]
.create(dto)	Crea instancia en memoria (sin guardar)	Entity
.save(entidad)	Guarda o actualiza en la BD	Entity
.preload({ id, ...datos })	Carga registro + mezcla nuevos datos (para update)	Entity undefined
.delete({ campo: valor })	Elimina por campo	DeleteResult

Ejemplo: Service con operaciones CRUD

■ src/employees/employees.service.ts

```
@Injectable()
export class EmployeesService {
  constructor(
    @InjectRepository(Employee)
    private employeeRepository: Repository<Employee>
  ) {}

  async create(dto: CreateEmployeeDto) {
    const employee = this.employeeRepository.create(dto); // instancia en memoria
    return await this.employeeRepository.save(employee); // guarda en BD
  }

  findAll() {
    return this.employeeRepository.find();
  }

  async update(id: string, dto: UpdateEmployeeDto) {
    const employee = await this.employeeRepository.preload({ employeeId: id, ...dto });
    if (!employee) throw new NotFoundException();
    return this.employeeRepository.save(employee);
  }

  remove(id: string) {
    this.employeeRepository.delete({ employeeId: id });
    return { message: 'employee deleted' };
  }
}
```

04 @nestjs/config

Variables de entorno con archivo .env

¿Qué es?

Permite leer un archivo **.env** y exponer sus valores como **process.env.VARIABLE**. Así los datos sensibles (contraseñas, claves, URLs) no se escriben directamente en el código.

Configuración (solo una vez en AppModule)

```
■ src/app.module.ts
import { ConfigModule } from '@nestjs/config';

@Module({
  imports: [
    ConfigModule.forRoot(), // lee el archivo .env automáticamente
    // ... resto de módulos
  ],
})
export class AppModule {}
```

Archivo .env

```
■ .env (en la raíz del proyecto, NO se sube al repositorio)
host=localhost
port=5432
pass=mi_password_segura
name=ocso_db
```

Uso en cualquier parte del código

```
■ src/app.module.ts
TypeOrmModule.forRoot({
  host: process.env.host,
  port: +(process.env.port || '5432'),
  password: process.env.pass,
  database: process.env.name,
})
```

Regla de oro: nunca hardcodear contraseñas o claves en el código. Siempre usar variables de entorno. Agrega `.env` al archivo `.gitignore` para que no se suba al repositorio.

05 class-validator + class-transformer

Validación automática de datos entrantes

¿Qué es?

class-validator verifica que los datos que llegan en el `@Body()` cumplan con las reglas definidas en el DTO. Si algo no cumple, NestJS responde automáticamente con un error 400 Bad Request.

Activación global en main.ts

```
■ src/main.ts
app.useGlobalPipes(new ValidationPipe({
  whitelist: true, // ignora campos no declarados en el DTO
  forbidNonWhitelisted: true, // error 400 si llegan campos extra
  transform: true, // convierte tipos automáticamente (string -> number)
}))
```

Decoradores de validación más usados

Decorador	Valida que...
@IsString()	El valor sea una cadena de texto
@IsEmail()	El valor tenga formato de correo electrónico
@IsNumber()	El valor sea un número
@IsBoolean()	El valor sea true o false
@MinLength(n)	El texto tenga al menos n caracteres
@MaxLength(n)	El texto tenga máximo n caracteres
@IsOptional()	El campo puede no venir en el request
@IsIn(['a','b'])	El valor sea uno de los permitidos
@IsObject()	El valor sea un objeto
@IsArray()	El valor sea un array

Ejemplo: DTO del usuario

■ `src/auth/dto/create-user.dto.ts`

```
import { IsEmail, IsIn, IsOptional, IsString, MinLength } from 'class-validator';

export class CreateUserDto extends User {
  @IsEmail()
  declare userEmail: string;

  @IsString()
  @MinLength(8) // mínimo 8 caracteres
  declare userPassword: string;

  @IsOptional() // si no viene, toma el default de la entidad
  @IsIn(['Admin', 'Employee', 'Manager']) // solo estos valores son válidos
  declare userRoles: string[];
}
```

06 bcrypt

Hasheo seguro de contraseñas

¿Qué es?

bcrypt convierte una contraseña en texto plano a un hash irreversible. Aunque alguien robe la base de datos, no podrá obtener la contraseña original. Para verificar si una contraseña es correcta, se compara con el hash guardado.

Las dos operaciones esenciales

```
■ src/auth/auth.service.ts
import * as bcrypt from 'bcrypt';

// Al REGISTRAR un usuario: hashear la contraseña antes de guardar
registerUser(createUserDto: CreateUserDto) {
  createUserDto.userPassword = bcrypt.hashSync(
    createUserDto.userPassword,
    5 // salt rounds: cuántas veces se aplica el algoritmo (más = más seguro)
  );
  return this.userRepository.save(createUserDto);
}

// Al HACER LOGIN: comparar contraseña ingresada con el hash guardado
async loginUser(loginUserDto: LoginUserDto) {
  const user = await this.userRepository.findOne({
    where: { userEmail: loginUserDto.userEmail }
  });
  if (!user) throw new UnauthorizedException('Credenciales incorrectas');

  const match = await bcrypt.compare(
    loginUserDto.userPassword, // texto plano ingresado
    user.userPassword // hash guardado en BD
  );
  if (!match) throw new UnauthorizedException('No estás autorizado');
  // ... generar token
}
```


Nunca se puede 'desencriptar' un hash de bcrypt. Para verificar, bcrypt aplica el mismo proceso a la contraseña ingresada y compara los resultados. Por eso se usa `bcrypt.compare()` y no una comparación directa.

07 JWT — JSON Web Tokens

Autenticación sin estado con tokens

¿Qué es?

JWT es un estándar para crear tokens de autenticación. El servidor genera un token firmado con datos del usuario (payload). El cliente lo guarda y lo envía en cada request para identificarse, sin que el servidor guarde sesiones en memoria.

Flujo completo de autenticación

1	Cliente envía email + password a POST /auth/login
2	Backend verifica credenciales con bcrypt
3	Backend genera un token JWT con el payload del usuario
4	Cliente guarda el token (localStorage o memoria)
5	En cada request protegida: Authorization: Bearer &token&
6	AuthGuard verifica el token y permite o rechaza el acceso

Configuración del módulo JWT

```
■ src/auth/auth.module.ts
JwtModule.register({
  secret: JWT_KEY, // clave secreta para firmar los tokens
  signOptions: { expiresIn: EXPIRES_IN }, // tiempo de expiración: '120s', '1h', '7d'
  global: true // disponible en toda la app sin re-importar
})
```

src/auth/constants/jwt.constants.ts

```
■ src/auth/constants/jwt.constants.ts
export const JWT_KEY = 'ADSWFSDAWDA12312' // en producción: process.env.JWT_SECRET
export const EXPIRES_IN = '120s' // el token expira en 2 minutos
```

Generación del token en el Service

■ src/auth/auth.service.ts

```
constructor(  
  @InjectRepository(User) private userRepository: Repository<User>,,  
  private jwtService: JwtService, // inyectado del JwtModule global  
) {}  
  
async loginUser(loginUserDto: LoginUserDto) {  
  // ... verificación con bcrypt ...  
  
  const payload = {  
    userEmail: user.userEmail,  
    userPassword: user.userPassword,  
    userRoles: user.userRoles, // roles incluidos en el token  
  };  
  const token = this.jwtService.sign(payload);  
  return token; // el cliente guarda este token  
}
```

08 Guards

Protección de rutas — AuthGuard y RolesGuard

¿Qué son?

Los Guards son clases que implementan **CanActivate** e interceptan cada request antes de que llegue al controlador. Retornan **true** para permitir el acceso o lanzan una excepción para rechazarlo.

Orden de ejecución

■ Flujo de una request protegida

Request

↓

AuthGuard — ¿El token existe y es válido? Si no: 401 Unauthorized

↓ Si sí: guarda el payload en request.user

RolesGuard — ¿El usuario tiene el rol requerido? Si no: 403 Forbidden

↓

Controller — Procesa la request normalmente

AuthGuard — Verifica el token JWT

■ src/auth/guards/auth.guards.ts

```
@Injectable()
export class AuthGuard implements CanActivate {
  constructor(private jwtService: JwtService) {}

  private extractTokenFromHeader(request: Request): string | undefined {
    const [type, token] = request.headers.authorization?.split(' ') ?? [];
    return type === 'Bearer' ? token : undefined;
  }

  async canActivate(context: ExecutionContext): Promise<boolean> {
    const request = context.switchToHttp().getRequest();
    const token = this.extractTokenFromHeader(request);
    if (!token) throw new UnauthorizedException();

    const payload = await this.jwtService.verifyAsync(token, { secret: JWT_KEY });
    request['user'] = payload; // guarda los datos del usuario en la request
    return true;
  }
}
```

RolesGuard — Verifica el rol del usuario

■ src/auth/guards/roles.guards.ts

```
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    // Lee los roles requeridos del decorador @Roles en el handler
    const roles = this.reflector.get(Roles, context.getHandler());
    if (!roles) return true; // si no hay @Roles, permite el acceso

    const request = context.switchToHttp().getRequest();
    const user: User = request.user; // viene del AuthGuard
    return this.matchRoles(roles, user.userRoles);
  }

  matchRoles(roles: string[], userRoles: string[]): boolean {
    return userRoles.some(role => roles.includes(role));
  }
}
```

09 Decoradores Personalizados

Simplificando el código del controlador

¿Para qué?

Los decoradores personalizados agrupan lógica repetida en un solo decorador reutilizable. Así el controlador queda limpio y legible.

@Roles — Guarda metadata de roles requeridos

```
■ src/auth/decorators/roles.decorator.ts
import { Reflector } from '@nestjs/core';

// Crea un decorador que guarda metadata (los roles requeridos)
// en el handler del controlador para que RolesGuard lo lea
export const Roles = Reflector.createDecorator<string[]>();
```

@Auth — Combina Guards y Roles en uno

```
■ src/auth/decorators/auth.decorator.ts
import { applyDecorators, UseGuards } from '@nestjs/common';

// En lugar de escribir @Roles + @UseGuards en cada endpoint,
// se usa solo @Auth(ROLES.MANAGER)
export const Auth = (...roles: ROLES[]) => {
  roles.push(ROLES.ADMIN); // Admin siempre tiene acceso
  return applyDecorators(
    Roles(roles),
    UseGuards(AuthGuard, RolesGuard)
  );
};
```

@UserData — Extrae el usuario de la request

■ src/auth/decorators/user.decorator.ts

```
import { createParamDecorator, ExecutionContext } from '@nestjs/common';

// Crea un decorador de parámetro personalizado
// que extrae el usuario que guardó AuthGuard en request.user
export const UserData = createParamDecorator(
  (data: unknown, ctx: ExecutionContext) => {
    const request = ctx.switchToHttp().getRequest();
    return request.user;
  }
);

// Uso en el controlador:
// @Get('profile')
// getProfile(@UserData() user: User) { return user; }
```

Uso final en el controlador

■ src/employees/employees.controller.ts

```
// Sin decoradores personalizados (código repetitivo):
// @Roles([ROLES.MANAGER, ROLES.ADMIN])
// @UseGuards(AuthGuard, RolesGuard)
// @Post()

// Con @Auth (limpio y reutilizable):
@Auth(ROLES.MANAGER) // solo Manager y Admin pueden crear employees
@Post()
create(@Body() dto: CreateEmployeeDto) {
  return this.employeesService.create(dto);
}

@Auth(ROLES.ADMIN) // solo Admin puede eliminar
@Delete(':id')
remove(@Param('id') id: string) {
  return this.employeesService.remove(id);
}
```

010 Pipes

Transformación y validación de parámetros

¿Qué son?

Los Pipes transforman o validan datos antes de que lleguen al método del controlador. NestJS incluye pipes predefinidos para los casos más comunes.

Los dos pipes usados en este proyecto

Pipe	Dónde	Para qué
ValidationPipe	Global en main.ts	Valida el @Body() contra el DTO con class-validator
ParseUUIDPipe	Por parámetro en el controller	Valida que @Param('id') sea un UUID válido

ValidationPipe — Validación global

```
■ src/main.ts
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,
  forbidNonWhitelisted: true,
  transform: true,
}))

// Resultado: si el @Body() no cumple las reglas del DTO,
// NestJS responde 400 automáticamente sin llegar al controlador
```

ParseUUIDPipe — Validación de parámetros de ruta

```
■ src/employees/employees.controller.ts
@Get(':id')
findOne(
  @Param('id', new ParseUUIDPipe({ version: '4' })) id: string
  // ↑ valida que ':id' sea un UUID v4 válido
  // Si no es válido: responde 400 automáticamente
) {
  return this.employeesService.findOne(id);
}
```


Sin ParseUUIDPipe, si alguien envía un ID inválido como /employees/hola, TypeORM lanzaría un error interno 500. Con el pipe, NestJS lo rechaza antes con un 400 Bad Request descriptivo.

011 Swagger

Documentación automática e interactiva

¿Qué es?

Swagger genera automáticamente una interfaz web en **/api** que muestra todos los endpoints, permite probarlos y sirve como documentación para el equipo frontend. Basta con agregar decoradores al código.

Configuración en main.ts

```
■ src/main.ts
const config = new DocumentBuilder()
  .setTitle('Ocso API')
  .setDescription('Api for ocso management')
  .setVersion('0.9')
  .addBearerAuth() // agrega campo de token JWT en la UI
  .build();

const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('api', app, document);
// UI disponible en: http://localhost:3000/api
```

Decoradores de Swagger

Decorador	Para qué
@ApiTags('nombre')	Agrupar endpoints bajo una etiqueta en la UI
@ApiProperty()	Documenta una propiedad del DTO (la muestra en el schema)
@ApiResponse({ status, description })	Documenta un posible tipo de respuesta
applyDecorators(ApiResponse(...))	Agrupar respuestas comunes en un decorador reutilizable

@ApiAuth — Decorador reutilizable con respuestas comunes

■ `src/auth/decorators/api.decorator.ts`

```
export const ApiAuth = () => {  
  return applyDecorators(  
    ApiResponse({ status: 401, description: 'Missing or invalid token' }),  
    ApiResponse({ status: 403, description: 'Missing role' }),  
    ApiResponse({ status: 500, description: 'Server error' }),  
  );  
};  
  
// Uso en el controlador:  
@ApiAuth() // documenta las respuestas 401, 403 y 500 de todos los endpoints  
@ApiTags('Auth')  
@Controller('auth')  
export class AuthController { ... }
```

012 Multer — File Upload

Recepción de archivos en los endpoints

¿Qué es?

Multer es un middleware para manejar **multipart/form-data**, el formato que se usa para subir archivos. NestJS lo integra a través de **FileInterceptor**.

Uso en el Controller

```
■ src/employees/employees.controller.ts
import { FileInterceptor } from '@nestjs/platform-express';
import { UseInterceptors, UploadedFile } from '@nestjs/common';

@Post('upload')
@UseInterceptors(FileInterceptor('file')) // 'file' = nombre del campo en el formulario
uploadPhoto(@UploadedFile() file: Express.Multer.File) {
  console.log(file.originalname); // nombre del archivo
  console.log(file.mimetype); // tipo: image/jpeg, application/pdf...
  console.log(file.size); // tamaño en bytes
  console.log(file.buffer); // contenido del archivo en memoria
  return 'archivo recibido';
}
```

FileInterceptor('file') lee el campo 'file' del form-data. El nombre debe coincidir exactamente con el campo que envía el cliente. Para guardar el archivo en disco, se configura diskStorage en las opciones de Multer.

0 Orden Paso a Paso para Nuevos Proyectos

Guía de referencia rápida

1

Crear el proyecto

`nest new nombre-proyecto`

2

Instalar dependencias

`npm install @nestjs/typeorm typeorm pg @nestjs/config class-validator class-transformer @nestjs/jwt bcrypt @nestjs/swagger uuid`

3

Crear el archivo .env

host, port, pass, name (datos de conexión a PostgreSQL)

4

Configurar AppModule

`ConfigModule.forRoot()` + `TypeOrmModule.forRoot()` con datos del .env

5

Crear recursos base

`nest g resource nombre` — genera module, controller, service y DTOs

6

Definir la Entidad

Decoradores `@Entity`, `@Column`, relaciones `@OneToOne`/`@ManyToOne`/`@OneToMany`

7

Completar los DTOs

Agregar decoradores de `class-validator` a cada propiedad

8

Registrar la entidad en el módulo

`TypeOrmModule.forFeature([Entidad])` en el imports del módulo

9

Completar el Service

CRUD con el Repository: `create/save`, `find`, `preload/save`, `delete`

10

Completar el Controller

Rutas @Get/@Post/@Patch/@Delete con @Body, @Param, @ParseUUIDPipe

11

Implementar Auth

Entidad User + bcrypt al registrar + JwtModule + generar token en login

12

Crear AuthGuard

Verifica el token JWT en el header y guarda el payload en request.user

13

Crear RolesGuard

Lee los roles del @Roles decorator y verifica contra user.userRoles

14

Crear decoradores personalizados

@Roles (Reflector.createDecorator) + @Auth (applyDecorators) + @UserData

15

Aplicar @Auth() en controladores

@Auth(ROLES.MANAGER) sobre los métodos o toda la clase

16

Configurar Swagger

DocumentBuilder + SwaggerModule.setup en main.ts. Agregar @ApiTags, @ApiProperty

17

Configurar ValidationPipe global

app.useGlobalPipes(new ValidationPipe({ whitelist, forbidNonWhitelisted, transform })))

18

Probar la API

Abrir <http://localhost:3000/api> y probar cada endpoint con Swagger UI

0 Resumen del Stack

Tecnologías y su rol

Tecnología	Rol en el proyecto	Cuándo se aplica
TypeScript	Lenguaje base con tipado estático	Siempre — es la base de todo
NestJS	Framework: módulos, rutas, DI	Desde el inicio
TypeORM	ORM para PostgreSQL	Al definir entidades y hacer CRUD
@nestjs/config	Variables de entorno (.env)	Antes de conectar la BD
class-validator	Validar datos entrantes (DTOs)	Al definir los DTOs
bcrypt	Hashear contraseñas	Al implementar registro/login
JWT	Autenticación con tokens	Al implementar login y rutas protegidas
Guards	Proteger rutas con lógica	Después de implementar JWT
Decoradores custom	Simplificar el controlador	Después de tener Guards
Pipes	Validar parámetros de ruta	Al definir rutas con @Param
Swagger	Documentación interactiva	Al finalizar los endpoints
Multer	Recibir archivos	Cuando se necesita upload de archivos